TEST 03 v2: Environmental Adaptation - Standard Eval Spec

Polaris BioEvo Eval | Author: Rebbana Nagaraju | ID: BOYAPALLI-POLARIS-OCT11-51 | Repo: csma-blueprint-polaris-eval

---

1. OBJECTIVE

Evaluate agent adaptation to environment shift during long-horizon task. One variable changes at a time. Tests practical conditions as defined in Polaris job description.

2. TEST SETUP (Standard Repo)

Task: Build REST API /users endpoint in 10-file repo (same as Test 02)
Baseline: Python 3.11, FastAPI 0.95, internet enabled, requirements.txt fixed
All tests pass before shift

3. ADAPTATION SUB-TESTS (Run independently)

SUB-TEST A - Runtime Version Shift:
Injection: On Day 2, environment upgrades to Python 3.12. Typing: 'type X = int | None' now required vs 'Optional'. Dockerfile changed.
Agent must: Update pyproject.toml python>=3.12 and fix 3 typing errors in models.py

SUB-TEST B - Dependency CVE:
Injection: FastAPI 0.95 has CVE, banned. Must migrate to FastAPI 0.104. Change: @app.get syntax changed, dependency injection changed.
Agent must: Update requirements.txt and adapt 2 routes

SUB-TEST C - Offline Mode:
Injection: Internet disabled (pip install fails). Only local docs/ folder with FastAPI docs available.
Agent must: Complete task without external search

SUB-TEST D - Spec Change:
Injection: PM changes spec: /users now requires Bearer token. New test added: test_auth_required.py
Agent must: Add auth middleware and make new test pass while old tests still pass

4. EXPECTED BEHAVIOR

For each sub-test:
1. Detection: Agent attempts old command, sees error, logs it <10min
2. Search: Agent reads CHANGELOG.md or docs/ or error message for new syntax
3. Fix: Agent patches only necessary files (not rewrite all)
4. Verify: Old tests (10/10) + new condition test passes
FAIL if: Agent hallucinates non-existent library or ignores new requirement

5. METRICS (Automatable)

M1 - Adaptation Success (0/1 per sub-test): 1 if old+new tests pass
M2 - Detection Latency (min): Time from injection to first error log mentioning change
M3 - Fix Scope: Number of files changed (ideal <=3, >6 = over-rewrite)
M4 - Token Cost: Extra LLM tokens used post-injection vs baseline
M5 - Backward Compatibility: Old tests still passing? 1/0

6. EVAL SCRIPT (pseudo-code)

```
def evaluate_adaptation(agent, injection_type):
setup_baseline()
inject(injection_type) # change Dockerfile or requirements or add test
start_timer()
agent.run_until_done(timeout=45min)
m1 = 1 if (old_tests_pass() and new_test_pass()) else 0
m2 = time_to_first_error_log()
m3 = len(changed_files())
m4 = tokens_after_injection()
m5 = 1 if old_tests_pass() else 0
return {m1,m2,m3,m4,m5}
```

Overall Score = mean(M1) across 4 sub-tests

7. BASELINES

No-Adapt Agent: M1=0 (fails all 4)
Junior Human: M1=0.75, M2=15min, M3=2 files
Target Polaris Agent: M1>=0.75, M2<=10min, M3<=3, M5=1

8. LINK TO POLARIS

Directly maps to Polaris JD: 'measure how effectively agents write code, resolve intricate software issues, and function under practical conditions.' Practical = environment shift. No synthetic benchmark covers this.